



DIVISION OF PHYSICAL SCIENCES AND MATHEMATICS

College of Arts and Sciences | University of the Philippines Visayas

Miagao, Iloilo, 5023, Philippines | Tel. No. (033) 513-7020

Email Address: psm.upvisayas@up.edu.ph

COURSE MATERIAL

When Storage Exists

CMSC 124 Activity 8

Prepared by: Rene Andre Bedonia Jocsing

Published: September 21, 2026

Deadline: September 25, 2026

The identifier `box`, its stored reference, and the heap object reached through that reference aren't the same thing. Today you'll put each one on a timeline. The program's behavior becomes easier to trace once every storage cell and object has a clear beginning and end.

In this activity's pseudocode, `new Cell(v)` allocates one `Cell` object on the heap, sets its `value` field to `v`, and returns a reference to that object. The reference is a value that another variable can copy, pointing to the same object.

The figures below separate them one at a time.

One identifier, one cell, one object

Three separate things with three separate lifetimes, taken one at a time.

The identifier names a cell

The identifier `box` is a name in the source text. What it names is one stack cell, created when the call begins.



Figure 1: The identifier names a cell

The cell holds a reference

After `new` runs, the cell contains a reference written `H1`. The reference is a value stored in the cell, not the object itself.

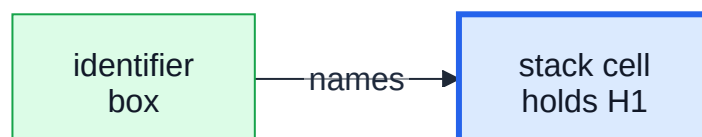


Figure 2: The cell holds a reference

The reference reaches the object

H1 is a third thing, sitting on the heap with its own lifetime. A second identifier can hold the same reference, and then both reach the one object.

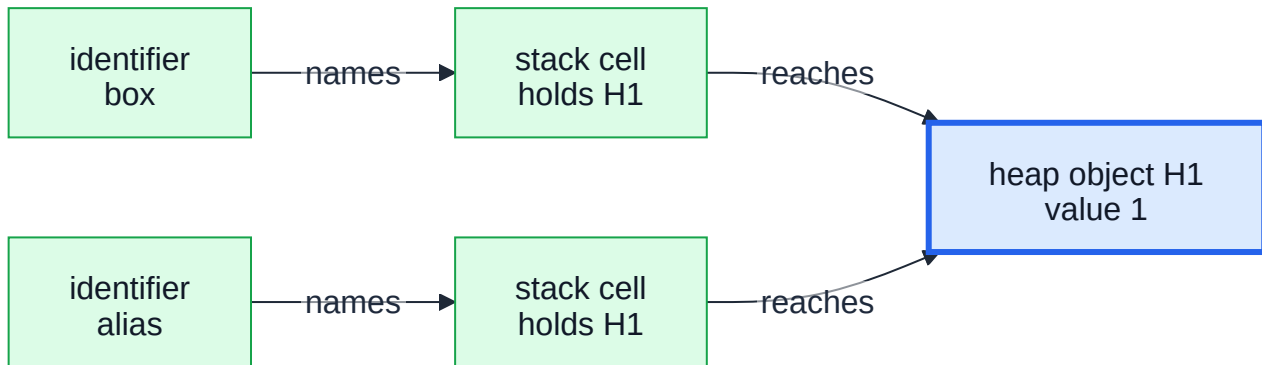


Figure 3: The reference reaches the object

The programs here are pseudocode, written to be read. Every rule they depend on is on this page.

You have 60 minutes for the two checkpoints. Keep each worked example beside the checkpoint it supports. Discussion with classmates and the instructor is allowed and encouraged throughout. This activity is worth 10 points: six for the lifetime construction, four for the rules that explain it.

The Storage Rules

A **binding** connects an entity to an attribute. An identifier can be bound to a type, a storage cell, or a value. **Binding time** is when that connection is chosen. In today's pseudocode, variable types are fixed when the program is translated, while each call's local cells are bound at run time.

Those two don't cover every option. A binding can be settled at six common moments:

Binding time	The connection is chosen when	Example
Language design time	The language itself is specified	<code>*</code> is the multiplication operator
Language implementation time	The compiler or interpreter is built	<code>int</code> gets its range of representable values
Compile time	The source is translated	<code>delta</code> is fixed as an integer variable
Link time	Separately translated pieces are joined	A call to a library routine finds that routine's code
Load time	The program is placed in memory	<code>total</code> receives the address of its static cell
Run time	The program is executing	<code>box</code> receives the address that <code>new</code> returned

The earlier the binding, the less work remains for run time and the less the program can change while running. That trade drives most of the storage decisions below.

When the program executes `record(3)`, it **calls** the function `record` with argument 3. That call begins when control enters `record` and returns when control leaves it. Each call gets fresh cells for its parameter and local variables.

The `delete box` operation follows the reference stored in `box` and deallocates the reached heap object. It leaves all reference values unchanged, including the value still stored in `box`.

Use these storage categories:

Category	Allocation	Deallocation
Static	Before execution	When the program ends
Stack-dynamic	When a call begins	When that call returns
Explicit heap-dynamic	When <code>new</code> runs	When <code>delete</code> runs

The labels **l-value** and **r-value** come from the two sides of an assignment. Read them literally. In `box = alias`, the left side needs the l-value of `box`, which identifies the cell to update. The right side needs the r-value of `alias`, which is the value read from that cell.

Read the two sides of an assignment

The assignment first finds the target cell on the left, then reads the value produced on the right.

Find the l-value on the left

In `box = alias`, the left occurrence of `box` identifies the stack cell that the assignment will update.

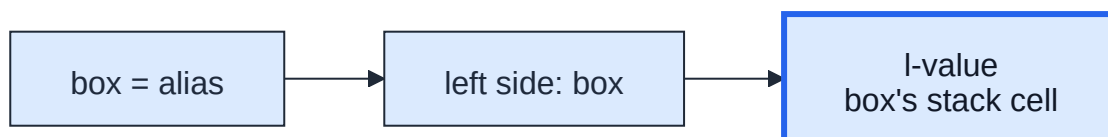


Figure 4: Find the l-value on the left

Read the r-value on the right

The right occurrence of `alias` reads H1 from its cell. The assignment copies H1 into `box's` cell.

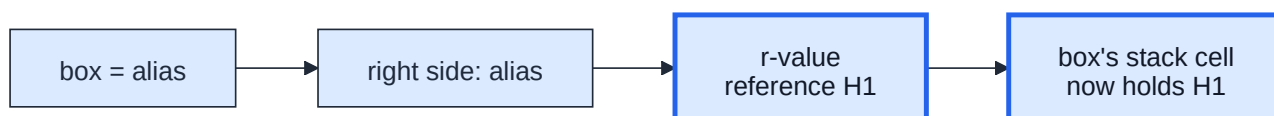


Figure 5: Read the r-value on the right

Two references are **aliases** when they reach the same object. An access that still points to a deallocated object is **dangling**. An object is **reachable** while some live access path leads to it. Under **explicit ownership**, the program must decide which code eventually deallocates each heap object.

Named Constants

A **named constant** is an identifier bound to a value once and never rebound. Its binding time determines whether the running program needs a storage cell for it.

```

1 static MAX = 100
2 static START = clock()
  
```

The translator can bind `MAX` at compile time because its value appears in the source. It can substitute 100 wherever `MAX` occurs and omit the cell. `START` depends on a run-time clock reading, so the program needs a cell for that value. Neither constant can be rebound after its first value is chosen.

Worked Example for Checkpoint 1

Suggested time: 38 minutes, including the worked example.

Trace one call in this smaller case:

```

1  static total = 1
2
3  inspect(reading):
4      delta = reading - total
5      box = new Cell(1)
6      alias = box
7      total = reading
8      delete box
9
10 inspect(4)

```

Event	Static storage	Stack storage	Heap storage
program begins	total = 1 allocated	none	none
inspect(4) begins	total = 1	allocate reading = 4, uninitialized delta, box, and alias	none
delta = reading - total	unchanged	delta = 3	none
new Cell(1)	unchanged	box receives reference H1	H1 allocated with value = 1
alias = box	unchanged	both reference cells contain H1	H1 still exists once
total = reading	total = 4	unchanged	unchanged
delete box	unchanged	box and alias contain dangling references	H1 deallocated
return	total = 4 remains	all call cells deallocated	H1 remains dead

The l-value of `box` is its stack cell. Its r-value after `new` is reference H1. H1 is a third object with its own lifetime. The rule says `delete` deallocates the reached heap object. It doesn't clear every reference that holds that address.

Erasing `alias` when `delete box` runs would require the runtime to find and rewrite all aliases, which isn't part of this language's contract. `alias` dangles from the deletion until the call returns.

The four figures below show one program at four moments. A box appears when its storage is allocated and turns red when that storage becomes invalid.

Storage begins and ends at different events

The variable cells and the heap object are separate entities even when the variables contain references to that object.

Allocate static storage

`total` exists before the call and begins with value 1.

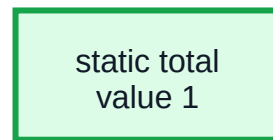


Figure 6: Allocate static storage

Enter the call and allocate the object

The call creates stack cells. `box` and `alias` contain references to one heap object.

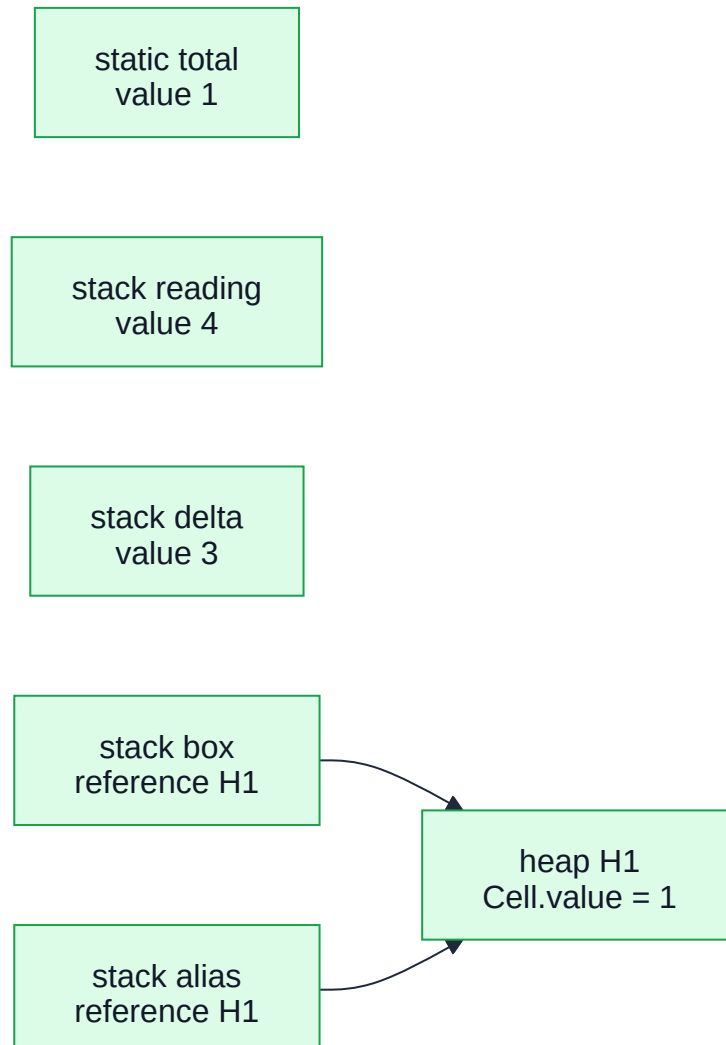


Figure 7: Enter the call and allocate the object

Delete the object, not the references

`delete box` ends H1's lifetime. The two stack cells still contain H1's address and are dangling until return.

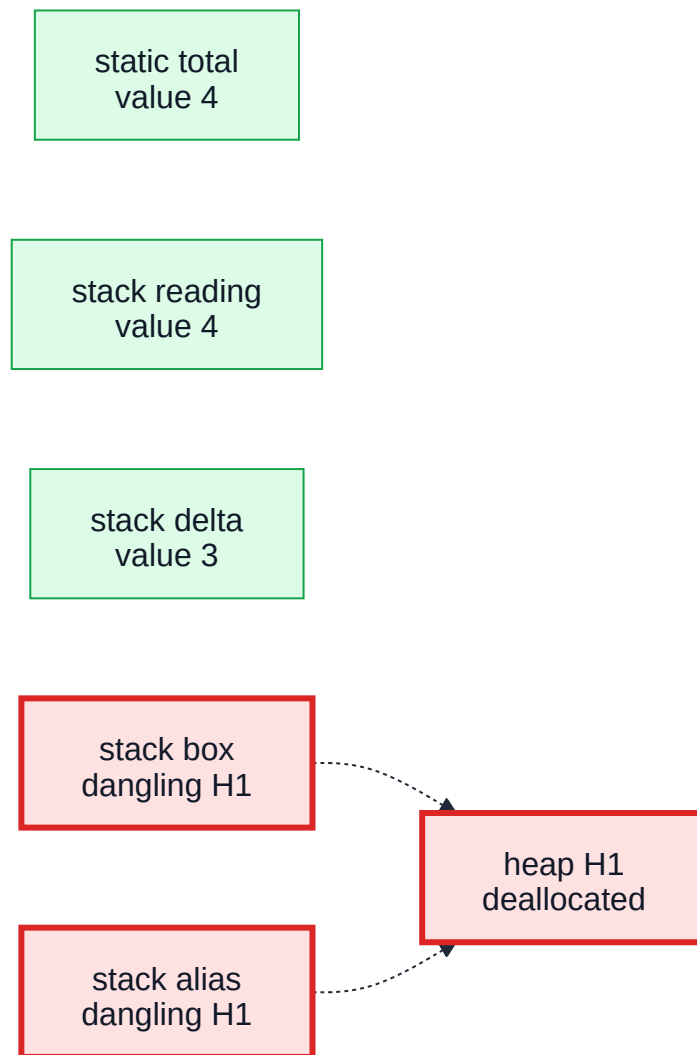


Figure 8: Delete the object, not the references

Return from the call

The stack cells disappear. Static total remains with value 4; H1 remains deallocated.

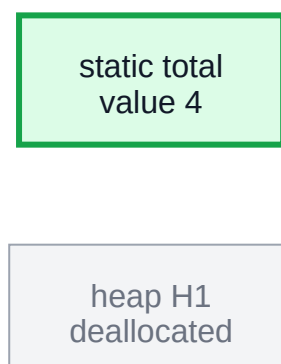


Figure 9: Return from the call

Checkpoint 1: Build the Lifetime Timeline (6 Points)**Running Case**

Execute this pseudocode in a language with explicit heap deallocation:

```
1  static visits = 0
2
3  record(reading):
4      delta = reading - visits
5      box = new Cell(delta)
6      alias = box
7      visits = reading
8      delete box
9
10 record(3)
11 record(8)
```

`Cell(v)` creates a heap object whose field `value` starts as `v`. Assigning `alias = box` copies the reference. Both variables then reach the same object. `delete box` deallocates that object. It doesn't rewrite either reference variable.

Apply these storage rules to the complete program above. `visits` is allocated before execution and lives until program end. Each parameter and local cell is allocated on call entry and deallocated on return. Each `Cell` object is allocated by `new` and deallocated by `delete`.

1. (Construction: 4 points) Draw a timeline for `visits`, both `reading` cells, both `delta` cells, the two `box` and `alias` pairs, and both heap objects. Mark allocation, every value change, deallocation, and each call return. Show the value of `visits` at every marked event.

2. (Explanation: 2 points) Answer both parts.

- Classify each item as static, stack-dynamic, or explicit heap-dynamic, then explain why `box` and the object it reaches have different lifetimes.
- Give the exact interval during which `alias` is dangling, with the event that starts it and the event that ends it.

Copy a Reference Past Return

Worked Example for Checkpoint 2

Suggested time: 22 minutes, including the worked example.

Trace this complete program:

```

1 static kept
2
3 make():
4     temp = new Cell(7)
5     return temp
6
7 kept = make()

```

Follow the events in execution order:

1. `new Cell(7)` allocates H3, and the local cell `temp` receives reference H3.
2. `return temp` reads H3 from `temp` and makes that reference the function's return value.
3. Returning from `make` deallocates its stack cells, including `temp`.
4. The assignment writes the returned reference into the static cell `kept`.

Copy a reference past return

The reference value crosses the return boundary while the local stack cell ends with its call.

Create H3 and store its reference in temp

`new Cell(7)` allocates H3. The local cell `temp` contains reference H3 and reaches that object.



Figure 10: Create H3 and store its reference in temp

Evaluate return temp

The return expression reads reference H3 from `temp`. The separate return value now preserves that reference.

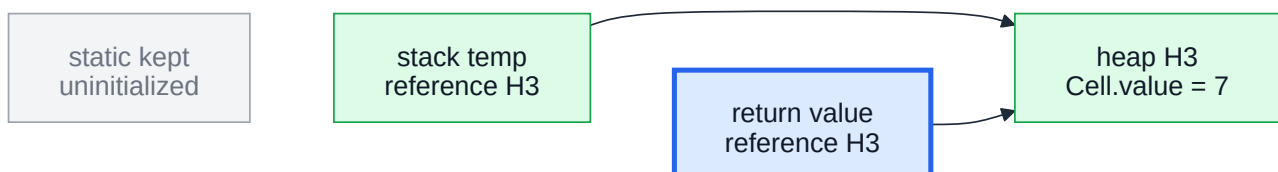


Figure 11: Evaluate return temp

After return, kept reaches H3

The final frame keeps the same four entities visible and marks the two values that no longer exist.

Return, then assign to kept

Return deallocates `temp`. The caller writes reference H3 into `kept`, which becomes the one remaining access path.

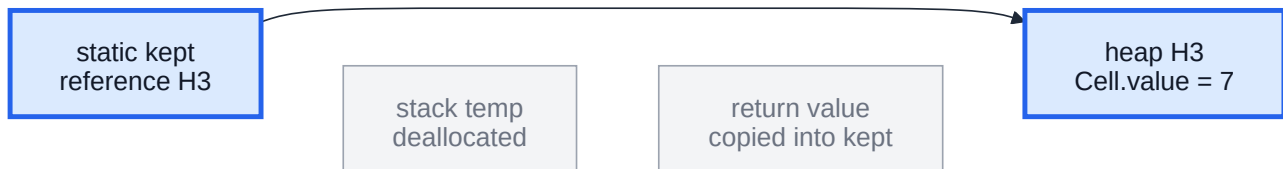


Figure 12: Return, then assign to kept

The reference value crosses the return boundary. The local cell does not. After the assignment, `kept` reaches H3 even though `temp` no longer exists.

Checkpoint 2: Follow the Returned Reference (4 Points)

Use this modified program. It has one call and does not execute `delete`:

```

1  static visits = 0
2  static saved
3
4  record(reading):
5      delta = reading - visits
6      box = new Cell(delta)
7      alias = box
8      visits = reading
9      return alias
10
11 saved = record(3)
  
```

Trace these events in order: allocation of H1, the copy into `alias`, evaluation of `return alias`, stack deallocation, and assignment to `saved`.

1. (Construction: 2 points) Redraw only the end of the first call. Show which stack cells disappear, which heap object remains, and what l-value and r-value belong to `saved`.

2. (Explanation: 2 points) Explain why returning the reference doesn't extend the stack lifetime of `alias`, yet still makes H1 reachable. State who must eventually deallocate H1 in this language.

End with H1 and the single live access path `saved` → H1 on the same timeline. The static cell `saved` is the l-value, and reference H1 is its stored r-value. The owner of `saved` must eventually deallocate H1 because reachability doesn't provide automatic ownership.